Atmospheric Motion Vector Project: Synthetic Model, Tracking, Quality Control, and Visualisation

Prepared for Ian Ajzenszmidt

21 May 2026

Contents

1 Purpose

This report documents a complete synthetic Atmospheric Motion Vector (AMV) demonstration. The project creates two synthetic cloud-image frames, advects cloud tracers through a known wind field, retrieves motion vectors by image-template tracking, refines the displacements to sub-pixel precision, assigns synthetic cloud-top height, applies quality control, and exports figures plus a vector CSV table.

2 Files included in this package

- `amv_report.tex`: this LaTeX report.
- `figures/*.png`: all generated model and visualisation images.
- `figures/animation.gif`: animated two-frame/cloud-motion visualisation if supported by the run.
- `data/amv_vectors.csv`: retrieved vector table.
- `data/run_summary.txt`: model run summary.
- `src/amv_model_demo.py`: complete Python AMV model.
- `src/amv_full_onefile_setup.sh`: one-file setup and run script.
- `src/setup_run_amv_demo.sh`: project setup script.
- `src/README_AMV_DEMO.md`: project readme.

3 Method summary

The demonstration uses a controlled synthetic experiment, so the retrieved vectors can be compared against a known reference wind field. The broad algorithm is:

1. Generate a synthetic cloud texture field.
2. Generate a spatially varying true horizontal wind field (u, v).
3. Advect the cloud texture to form a second satellite-like image.
4. Sample target boxes on a grid and search nearby windows in the second frame.
5. Score matches by normalised cross-correlation.
6. Estimate sub-pixel displacement by local parabolic peak fitting.
7. Apply quality-control filters using correlation score, displacement bounds, and consistency checks.
8. Assign height from a synthetic brightness-temperature/cloud-top relation.
9. Export vector data and visual diagnostics.

4 Run summary

```
ATMOSPHERIC MOTION VECTOR MODEL DEMO SUMMARY
============================================================
Image size:                  128 x 128 pixels
Pixel scale:                 4.00 km/pixel
Time step:                   10.00 minutes
Tracking window:             21 x 21 pixels
Search radius:               +/- 8 pixels
Grid step:                   16 pixels

Raw vectors attempted:       36
Basic QC passed:             5
Final QC passed:             5

Final QC statistics:
  Median vector error:       12.07 km/h
  Mean vector error:         16.76 km/h
  90th percentile error:     27.65 km/h
  Median NCC correlation:    0.985
  Median assigned height:    4.05 km

QC tests:
  min_corr >= 0.55
  corr margin >= 0.015
  speed <= 420.0 km/h
  neighbour consistency <= 80.0 km/h

Interpretation:
  This is an educational, synthetic AMV pipeline. A real operational
  AMV system would need navigation calibration, channel-specific cloud
  masking, radiative-transfer or NWP-assisted height assignment,
  parallax correction, objective quality indicators, and validation
  against radiosondes, aircraft, scatterometers, or NWP analysis winds.
```

5 Generated visualisations

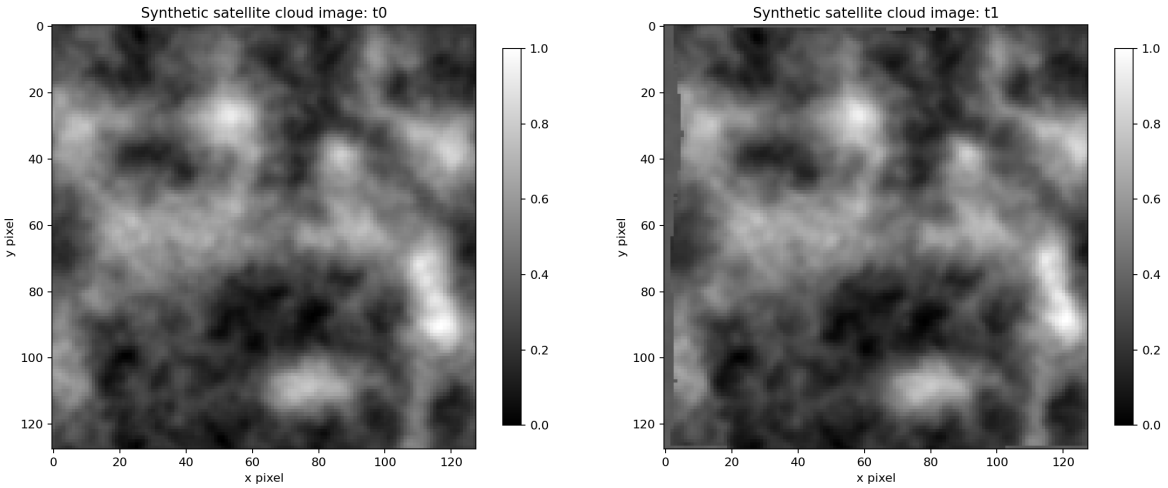

Figure 1: Synthetic satellite-like cloud fields at frame 0 and frame 1.

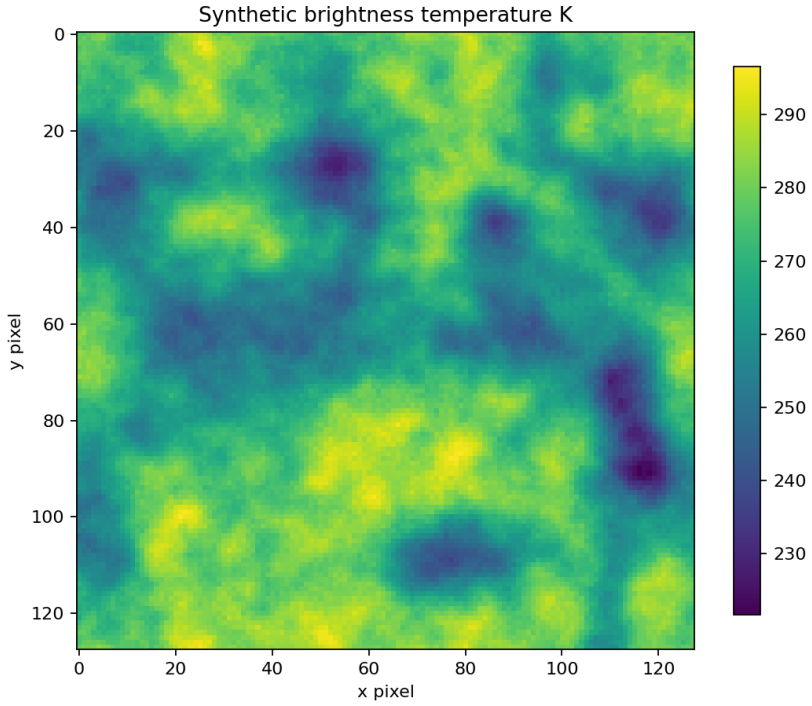

Figure 2: Synthetic brightness temperature field used for cloud-top height assignment.

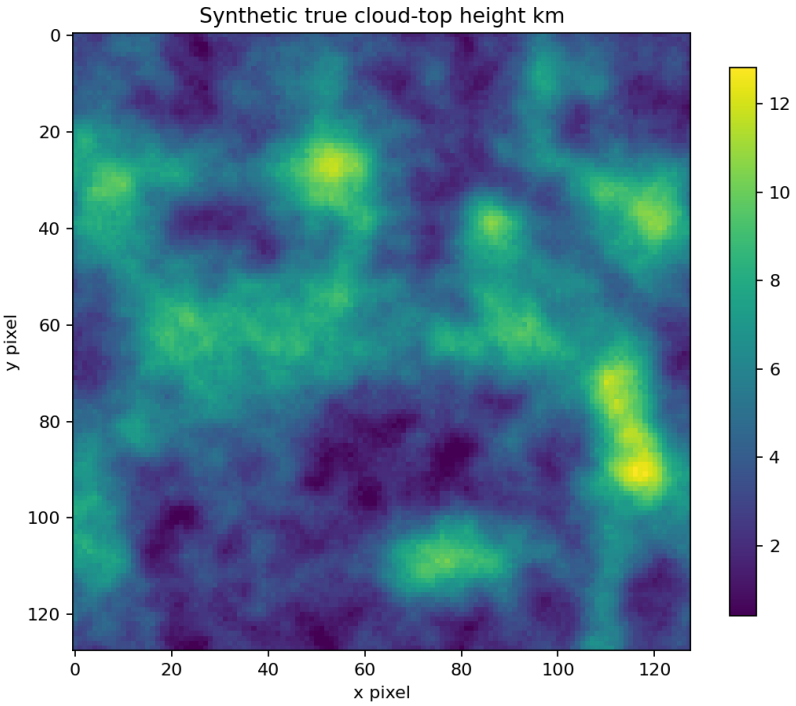

Figure 3: Synthetic true height field.

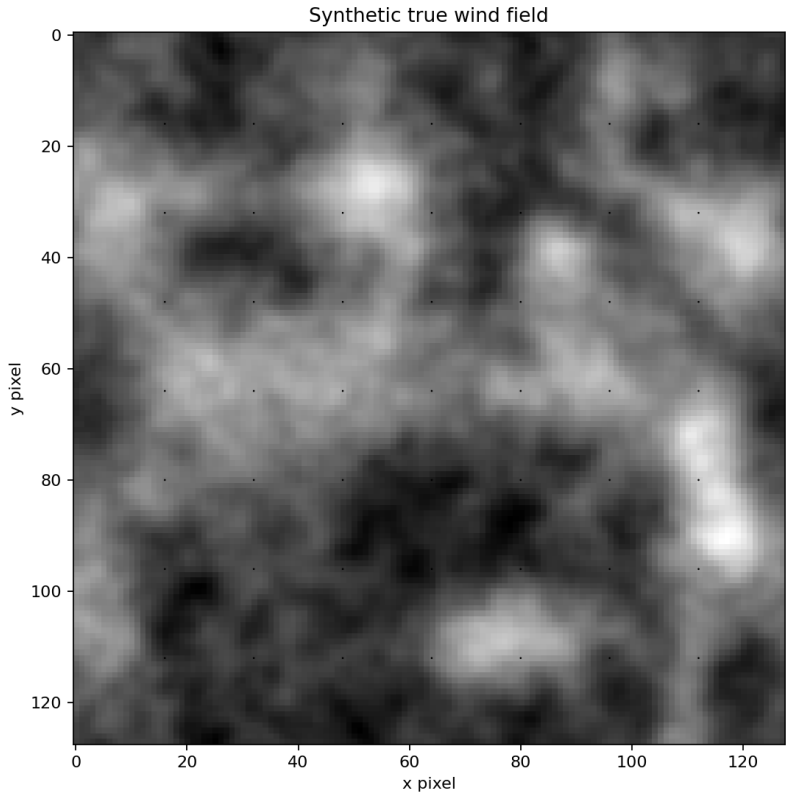

Figure 4: Reference wind field used to advect the synthetic cloud texture.

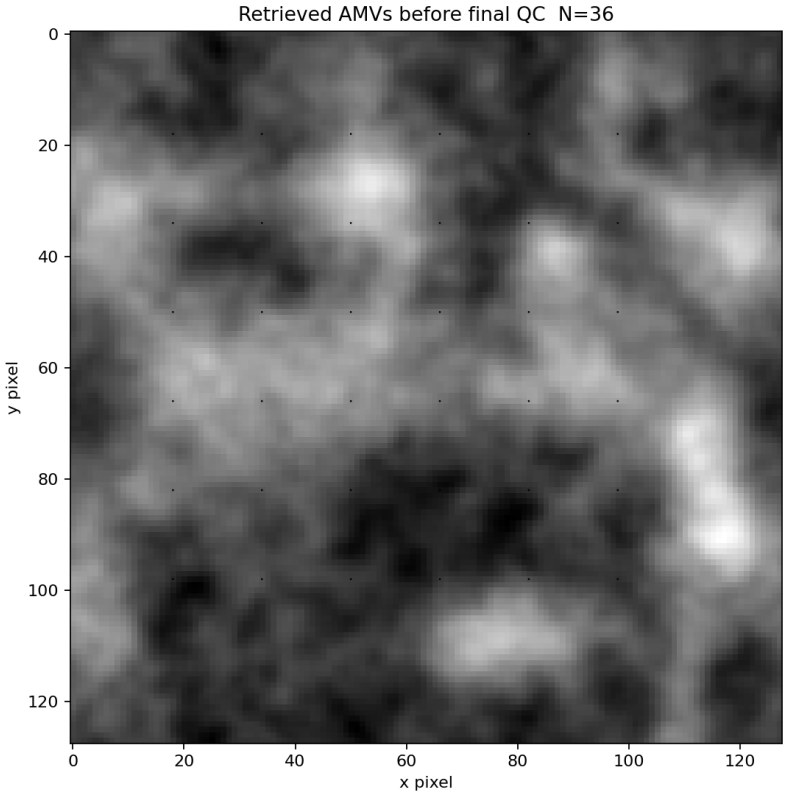

Figure 5: Raw retrieved AMVs before quality control.

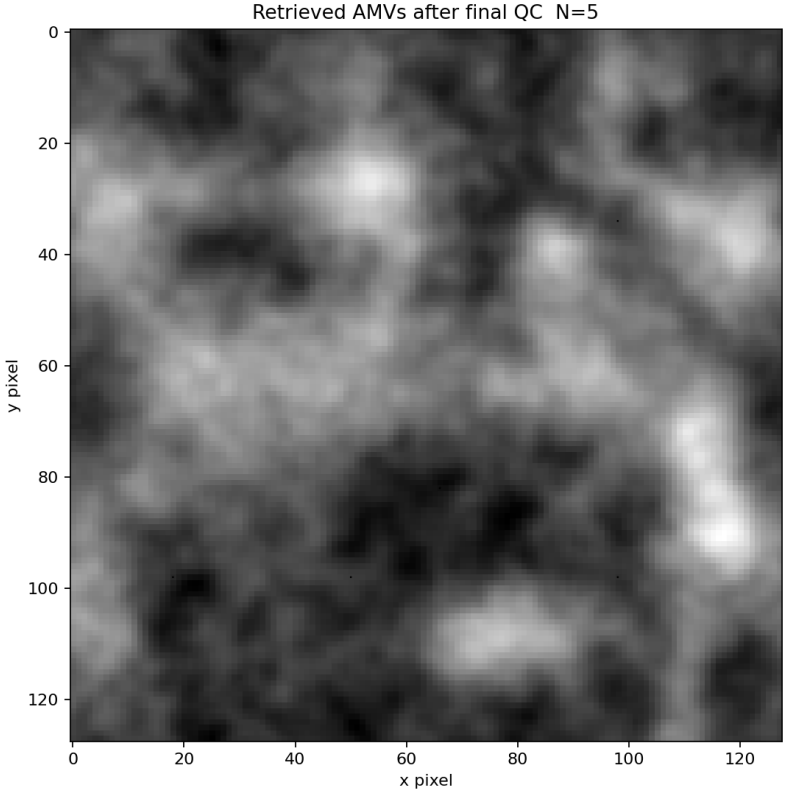

Figure 6: Retrieved AMVs after quality control.

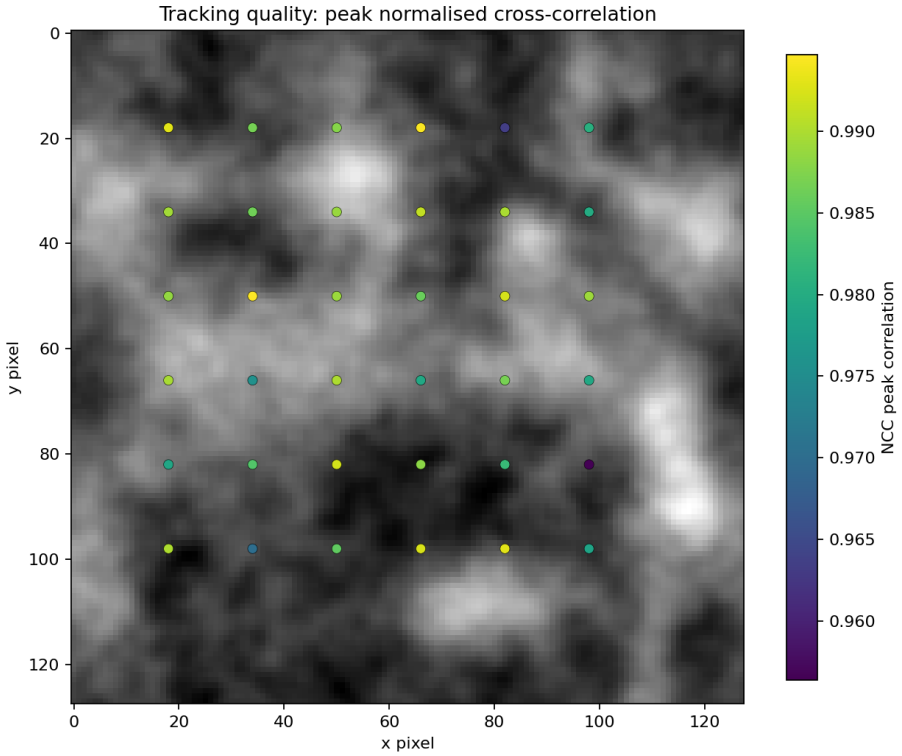

Figure 7: Tracking-quality diagnostic based on match strength and vector acceptance.

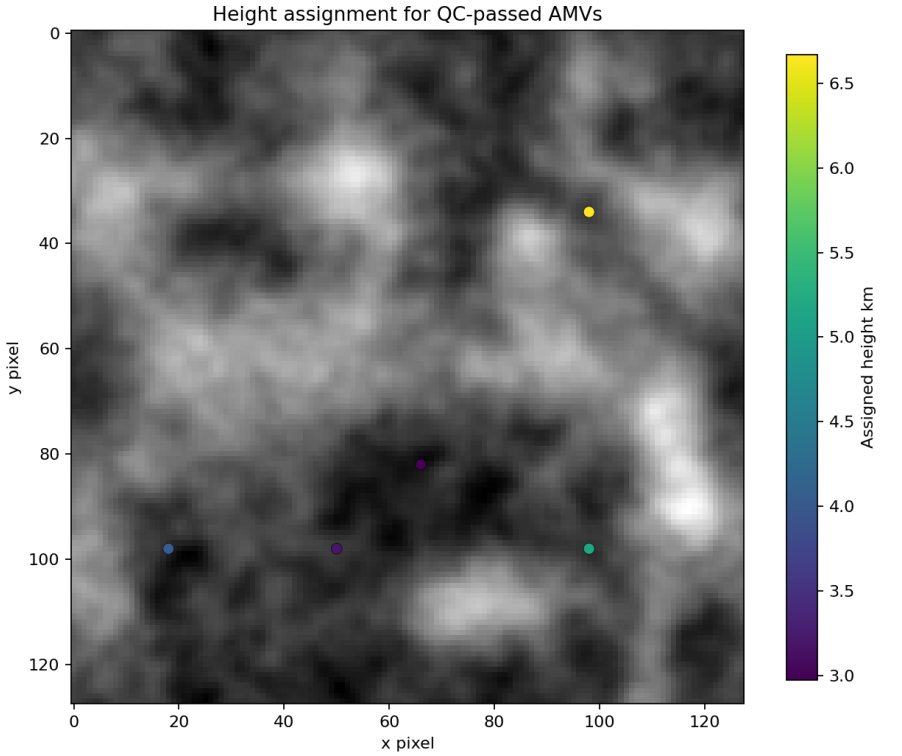

Figure 8: Assigned AMV heights derived from the synthetic thermal field.

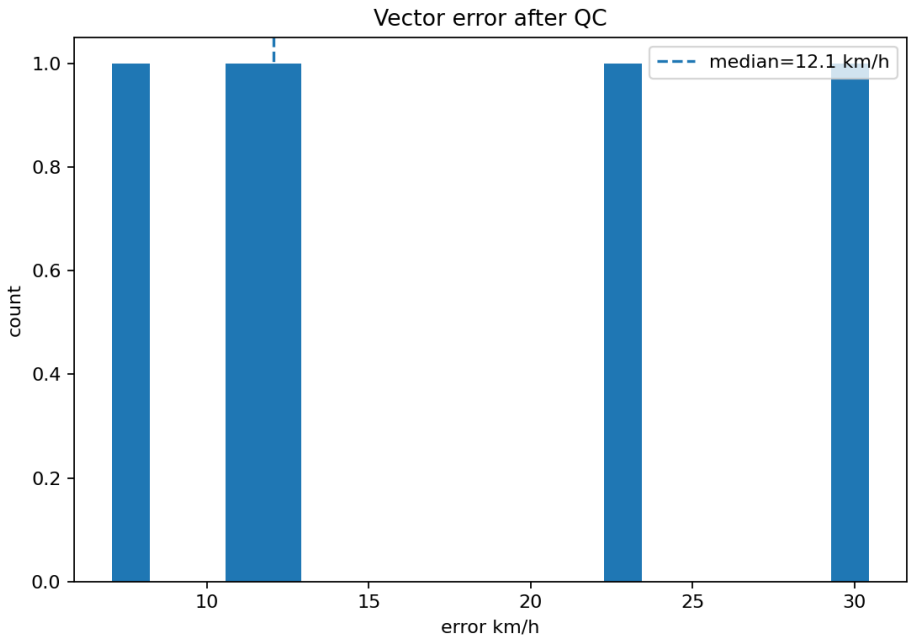

Figure 9: Distribution of vector error magnitude against the known synthetic wind field.

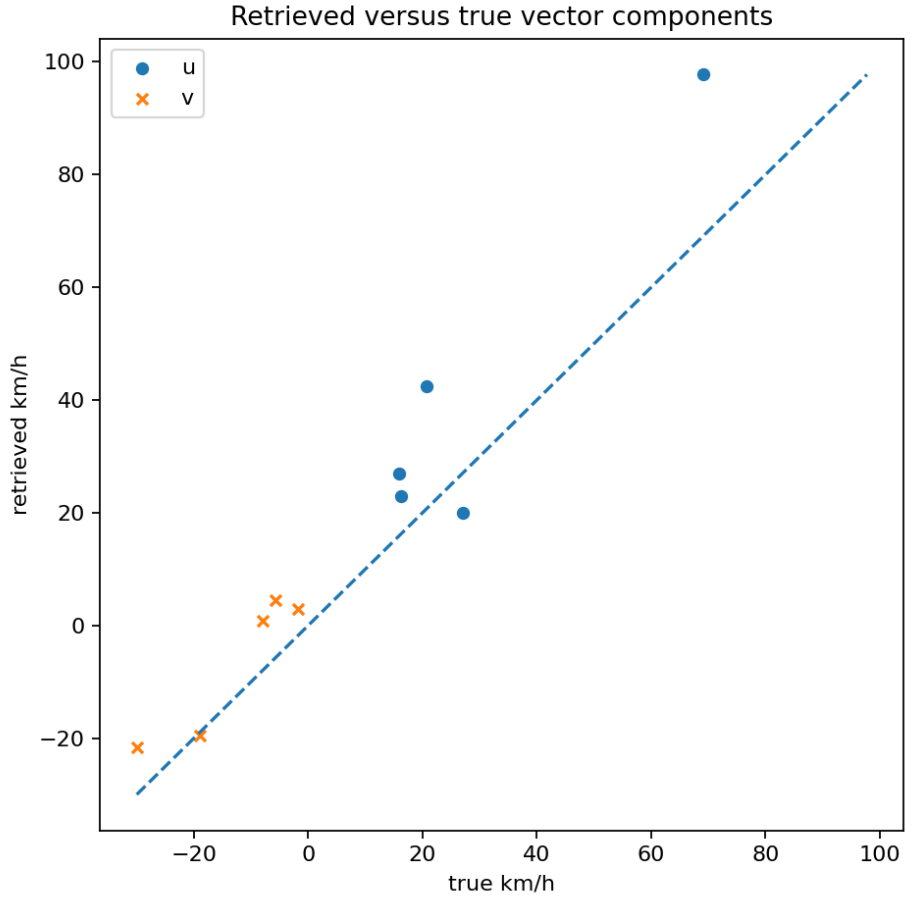

Figure 10: Scatter comparison of retrieved and true vector components.

6 CSV vector table: first rows

The full table is included as `data/amv_vectors.csv`. The first rows are reproduced below.

x	y	dx_pix	dy_pix	u_kmh	v_kmh	speed_kmh	corr	corr_margin	height_km	qc_f
18.000	18.000	2.518	-0.081	60.442	1.951	60.473	0.993	0.000	6.005	P
34.000	18.000	3.320	0.228	79.670	-5.467	79.858	0.987	0.006	5.052	P
50.000	18.000	3.700	0.157	88.808	-3.771	88.888	0.988	0.006	8.458	P
66.000	18.000	2.724	0.849	65.384	-20.375	68.485	0.994	0.004	5.252	P
82.000	18.000	2.939	0.400	70.533	-9.597	71.183	0.963	0.005	3.691	P
98.000	18.000	3.242	-0.434	77.806	10.404	78.499	0.981	0.005	6.085	P
18.000	34.000	3.527	-0.336	84.660	8.054	85.042	0.989	0.000	7.412	P
34.000	34.000	3.595	0.210	86.280	-5.045	86.427	0.986	0.002	5.561	P
50.000	34.000	4.286	0.237	102.876	-5.690	103.033	0.989	0.006	9.585	P
66.000	34.000	3.623	0.573	86.951	-13.752	88.031	0.991	0.001	7.764	P
82.000	34.000	4.003	0.206	96.080	-4.952	96.207	0.990	0.009	7.192	P
98.000	34.000	4.072	-0.188	97.737	4.511	97.841	0.980	0.024	6.672	T
18.000	50.000	3.672	-0.326	88.134	7.834	88.481	0.989	0.000	7.292	P
34.000	50.000	3.277	0.109	78.644	-2.619	78.687	0.995	0.004	7.517	P
50.000	50.000	3.712	0.200	89.082	-4.811	89.212	0.989	0.012	7.830	P
66.000	50.000	3.171	0.628	76.104	-15.082	77.584	0.986	0.005	6.547	P
82.000	50.000	4.072	-0.042	97.737	1.014	97.742	0.992	0.011	7.858	P
98.000	50.000	3.604	-0.162	86.503	3.881	86.590	0.989	0.004	7.490	P
18.000	66.000	3.536	-0.640	84.864	15.358	86.243	0.990	0.001	8.279	P
34.000	66.000	3.616	-0.089	86.781	2.137	86.807	0.976	0.006	8.128	P

7 Source listing: AMV Python model

```
#!/usr/bin/env python3
"""
AMV MODEL DEMO
Atmospheric Motion Vector simulation, tracking, quality control,
sub-pixel refinement, and height assignment.

This is a self-contained educational AMV demonstrator.
It does not download satellite data. It synthesizes two cloud images,
advects the cloud scene with a known wind field, then attempts to
recover motion vectors using template matching.

Outputs:
  output_amv/
    frame0.png
    frame1.png
    true_wind.png
    retrieved_vectors_raw.png
    retrieved_vectors_qc.png
    height_assignment.png
    error_histogram.png
    vector_scatter.png
    tracking_quality.png
    amv_vectors.csv
    run_summary.txt
    animation.gif      if Pillow is available

Coordinate convention:
  x increases eastward.
  y increases downward in image space.
  u is eastward wind km/h.
  v is northward wind km/h, so v = -dy * pixel_km / dt_hours.
"""

import argparse
import csv
```

```
import math
import os
import sys
from dataclasses import dataclass
from pathlib import Path

import numpy as np

import matplotlib
if "--show" not in sys.argv:
    matplotlib.use("Agg")
import matplotlib.pyplot as plt


@dataclass
class AMVConfig:
    size: int = 192
    pixel_km: float = 4.0
    dt_minutes: float = 10.0
    window: int = 25
    step: int = 16
    search_radius: int = 10
    seed: int = 42
    min_corr: float = 0.55
    min_margin: float = 0.015
    max_speed_kmh: float = 420.0
    consistency_kmh: float = 80.0
    outdir: str = "output_amv"
    show: bool = False
    make_gif: bool = True


def ensure_odd(n: int) -> int:
    return n if n % 2 == 1 else n + 1


def smooth2d(a: np.ndarray, passes: int = 5) -> np.ndarray:
    """Small dependency-free smoothing by repeated neighbour averaging."""
    out = a.astype(float).copy()
    for _ in range(passes):
        out = (
            out
            + np.roll(out, 1, axis=0)
            + np.roll(out, -1, axis=0)
            + np.roll(out, 1, axis=1)
            + np.roll(out, -1, axis=1)
            + np.roll(np.roll(out, 1, axis=0), 1, axis=1)
            + np.roll(np.roll(out, 1, axis=0), -1, axis=1)
            + np.roll(np.roll(out, -1, axis=0), 1, axis=1)
            + np.roll(np.roll(out, -1, axis=0), -1, axis=1)
        ) / 9.0
    return out


def normalize01(a: np.ndarray) -> np.ndarray:
    amin = float(np.nanmin(a))
    amax = float(np.nanmax(a))
    if amax - amin < 1e-12:
        return np.zeros_like(a, dtype=float)
    return (a - amin) / (amax - amin)


def make_cloud_scene(cfg: AMVConfig):
    rng = np.random.default_rng(cfg.seed)
    n = cfg.size
    y, x = np.mgrid[0:n, 0:n]

    # Multi-scale cloud texture.
    base = rng.normal(0.0, 1.0, (n, n))
    large = smooth2d(base, passes=22)
    medium = smooth2d(rng.normal(0.0, 1.0, (n, n)), passes=8)
    fine = smooth2d(rng.normal(0.0, 1.0, (n, n)), passes=2)
    cloud = 1.5 * large + 0.8 * medium + 0.25 * fine

    # Add Gaussian cloud shields and cellular patches.
```

```
    for _ in range(18):
        cx = rng.uniform(0, n)
        cy = rng.uniform(0, n)
        sx = rng.uniform(n * 0.035, n * 0.14)
        sy = rng.uniform(n * 0.035, n * 0.14)
        amp = rng.uniform(0.35, 1.4)
        cloud += amp * np.exp(-(((x - cx) / sx) ** 2 + ((y - cy) / sy) ** 2))

    # Add a frontal band to give elongated AMV targets.
    angle = math.radians(22.0)
    xr = (x - n * 0.5) * math.cos(angle) + (y - n * 0.5) * math.sin(angle)
    yr = -(x - n * 0.5) * math.sin(angle) + (y - n * 0.5) * math.cos(angle)
    band = np.exp(-((yr + 12 * np.sin(xr / 18.0)) / 13.0) ** 2)
    cloud += 0.8 * band

    cloud = normalize01(cloud)

    # Synthetic brightness temperature:
    # thicker/brighter cloud is colder and therefore higher.
    bt = 296.0 - 74.0 * cloud + rng.normal(0.0, 1.0, (n, n))
    height_km = np.clip((296.0 - bt) / 5.8, 0.4, 14.5)

    # True wind field in image displacement pixels per time step.
    # Higher cloud tops move faster and a little more zonally.
    hnorm = np.clip(height_km / 14.5, 0, 1)
    u_kmh = 25.0 + 115.0 * hnorm + 22.0 * np.sin(2 * np.pi * y / n)
    v_kmh = -18.0 + 55.0 * hnorm + 16.0 * np.cos(2 * np.pi * x / n)

    dt_hours = cfg.dt_minutes / 60.0
    dx_pix = u_kmh * dt_hours / cfg.pixel_km
    dy_pix = -v_kmh * dt_hours / cfg.pixel_km

    return cloud, bt, height_km, u_kmh, v_kmh, dx_pix, dy_pix


def bilinear_sample(img: np.ndarray, x: np.ndarray, y: np.ndarray, fill: float = 0.0) -> np.ndarray:
    h, w = img.shape
    x0 = np.floor(x).astype(int)
    y0 = np.floor(y).astype(int)
    x1 = x0 + 1
    y1 = y0 + 1

    wx = x - x0
    wy = y - y0

    valid = (x0 >= 0) & (x1 < w) & (y0 >= 0) & (y1 < h)
    out = np.full_like(x, fill, dtype=float)

    x0c = np.clip(x0, 0, w - 1)
    x1c = np.clip(x1, 0, w - 1)
    y0c = np.clip(y0, 0, h - 1)
    y1c = np.clip(y1, 0, h - 1)

    Ia = img[y0c, x0c]
    Ib = img[y0c, x1c]
    Ic = img[y1c, x0c]
    Id = img[y1c, x1c]

    interp = (
        Ia * (1 - wx) * (1 - wy)
        + Ib * wx * (1 - wy)
        + Ic * (1 - wx) * wy
        + Id * wx * wy
    )
    out[valid] = interp[valid]
    return out


def advect_scene(cloud0, bt0, dx_pix, dy_pix, seed=123):
    rng = np.random.default_rng(seed)
    n = cloud0.shape[0]
    yy, xx = np.mgrid[0:n, 0:n]

    # For each target pixel at time 1, sample the source pixel at time 0
    # from the approximate backward trajectory.
```

```
    src_x = xx - dx_pix
    src_y = yy - dy_pix
    fill = float(np.nanmedian(cloud0))
    cloud1 = bilinear_sample(cloud0, src_x, src_y, fill=fill)

    # Small evolution term: AMVs are harder when cloud features deform.
    evolution = 0.035 * smooth2d(rng.normal(0, 1, cloud0.shape), passes=2)
    cloud1 = normalize01(0.97 * cloud1 + evolution)

    bt1 = 296.0 - 74.0 * cloud1 + rng.normal(0.0, 1.1, cloud1.shape)
    return cloud1, bt1


def normxcorr(a: np.ndarray, b: np.ndarray) -> float:
    aa = a - float(np.mean(a))
    bb = b - float(np.mean(b))
    den = math.sqrt(float(np.sum(aa * aa)) * float(np.sum(bb * bb)))
    if den < 1e-12:
        return -1.0
    return float(np.sum(aa * bb) / den)


def parabolic_offset(c_minus: float, c0: float, c_plus: float) -> float:
    denom = c_minus - 2.0 * c0 + c_plus
    if abs(denom) < 1e-9:
        return 0.0
    delta = 0.5 * (c_minus - c_plus) / denom
    if not np.isfinite(delta):
        return 0.0
    return float(np.clip(delta, -1.0, 1.0))


def local_height_from_bt(bt: np.ndarray, x: int, y: int, half: int) -> float:
    patch = bt[y - half:y + half + 1, x - half:x + half + 1]
    # AMV height assignment often uses cold cloud information in the target.
    # Use 20th percentile brightness temperature: cold cloud top proxy.
    cold_bt = float(np.percentile(patch, 20))
    return float(np.clip((296.0 - cold_bt) / 5.8, 0.4, 14.5))


def track_amvs(frame0, frame1, bt0, cfg: AMVConfig):
    n = cfg.size
    win = ensure_odd(cfg.window)
    half = win // 2
    r = cfg.search_radius
    dt_hours = cfg.dt_minutes / 60.0

    rows = []
    centers = range(half + r, n - half - r, cfg.step)

    for cy in centers:
        for cx in centers:
            target = frame0[cy - half:cy + half + 1, cx - half:cx + half + 1]
            if float(np.std(target)) < 0.035:
                continue

            corr = np.empty((2 * r + 1, 2 * r + 1), dtype=float)
            for jj, dy in enumerate(range(-r, r + 1)):
                yy = cy + dy
                for ii, dx in enumerate(range(-r, r + 1)):
                    xx = cx + dx
                    cand = frame1[yy - half:yy + half + 1, xx - half:xx + half + 1]
                    corr[jj, ii] = normxcorr(target, cand)

            flat_index = int(np.argmax(corr))
            jmax, imax = np.unravel_index(flat_index, corr.shape)
            best = float(corr[jmax, imax])
            sorted_corr = np.sort(corr.ravel())
            second = float(sorted_corr[-2]) if sorted_corr.size >= 2 else -1.0
            margin = best - second

            # Integer displacement from frame0 centre to best frame1 centre.
            dx_int = imax - r
            dy_int = jmax - r
```

```
            # Sub-pixel correction using local parabolic peak.
            subx = 0.0
            suby = 0.0
            if 0 < imax < 2 * r:
                subx = parabolic_offset(corr[jmax, imax - 1], corr[jmax, imax], corr[jmax, imax + 1])
            if 0 < jmax < 2 * r:
                suby = parabolic_offset(corr[jmax - 1, imax], corr[jmax, imax], corr[jmax + 1, imax])

            dx = dx_int + subx
            dy = dy_int + suby

            u = dx * cfg.pixel_km / dt_hours
            v = -dy * cfg.pixel_km / dt_hours
            speed = math.sqrt(u * u + v * v)
            height = local_height_from_bt(bt0, cx, cy, half)

            qc_basic = (
                best >= cfg.min_corr
                and margin >= cfg.min_margin
                and speed <= cfg.max_speed_kmh
            )

            rows.append({
                "x": float(cx),
                "y": float(cy),
                "dx_pix": float(dx),
                "dy_pix": float(dy),
                "u_kmh": float(u),
                "v_kmh": float(v),
                "speed_kmh": float(speed),
                "corr": float(best),
                "corr_margin": float(margin),
                "height_km": float(height),
                "qc_basic": bool(qc_basic),
                "qc_consistency": False,
                "qc_final": False,
                "du_error_kmh": np.nan,
                "dv_error_kmh": np.nan,
                "vector_error_kmh": np.nan,
            })

    apply_consistency_qc(rows, cfg)
    return rows


def apply_consistency_qc(rows, cfg: AMVConfig):
    if not rows:
        return

    coords = np.array([[r["x"], r["y"]] for r in rows], dtype=float)
    u = np.array([r["u_kmh"] for r in rows], dtype=float)
    v = np.array([r["v_kmh"] for r in rows], dtype=float)
    basic = np.array([r["qc_basic"] for r in rows], dtype=bool)

    for i, row in enumerate(rows):
        if not row["qc_basic"]:
            row["qc_consistency"] = False
            row["qc_final"] = False
            continue

        dist = np.sqrt(np.sum((coords - coords[i]) ** 2, axis=1))
        nbr = (dist > 0) & (dist <= cfg.step * 1.6) & basic
        if np.count_nonzero(nbr) < 3:
            row["qc_consistency"] = True
            row["qc_final"] = True
            continue

        med_u = float(np.median(u[nbr]))
        med_v = float(np.median(v[nbr]))
        diff = math.sqrt((row["u_kmh"] - med_u) ** 2 + (row["v_kmh"] - med_v) ** 2)

        row["qc_consistency"] = bool(diff <= cfg.consistency_kmh)
        row["qc_final"] = bool(row["qc_basic"] and row["qc_consistency"])
```

```
def interpolate_truth_at_vectors(rows, u_true, v_true):
    n = u_true.shape[0]
    for row in rows:
        x = int(np.clip(round(row["x"]), 0, n - 1))
        y = int(np.clip(round(row["y"]), 0, n - 1))
        du = row["u_kmh"] - float(u_true[y, x])
        dv = row["v_kmh"] - float(v_true[y, x])
        row["u_true_kmh"] = float(u_true[y, x])
        row["v_true_kmh"] = float(v_true[y, x])
        row["du_error_kmh"] = float(du)
        row["dv_error_kmh"] = float(dv)
        row["vector_error_kmh"] = float(math.sqrt(du * du + dv * dv))


def rows_to_arrays(rows, final_only=False):
    selected = [r for r in rows if (r["qc_final"] or not final_only)]
    if not selected:
        return {}
    return {k: np.array([r[k] for r in selected]) for k in selected[0].keys()}


def save_csv(rows, path: Path):
    if not rows:
        return
    fieldnames = list(rows[0].keys())
    with path.open("w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        for r in rows:
            writer.writerow(r)


def save_image(img, path: Path, title: str, cmap="gray", colorbar=True):
    fig, ax = plt.subplots(figsize=(7, 6))
    im = ax.imshow(img, cmap=cmap, origin="upper")
    ax.set_title(title)
    ax.set_xlabel("x pixel")
    ax.set_ylabel("y pixel")
    if colorbar:
        fig.colorbar(im, ax=ax, shrink=0.82)
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def quiver_plot(frame, rows, path: Path, title: str, final_only=False, scale=2200):
    selected = [r for r in rows if (r["qc_final"] or not final_only)]
    fig, ax = plt.subplots(figsize=(8, 7))
    ax.imshow(frame, cmap="gray", origin="upper")
    if selected:
        x = np.array([r["x"] for r in selected])
        y = np.array([r["y"] for r in selected])
        u = np.array([r["u_kmh"] for r in selected])
        v = np.array([r["v_kmh"] for r in selected])
        # Convert north-positive v back to image-axis vector component for display.
        ax.quiver(x, y, u, -v, angles="xy", scale_units="xy", scale=scale, width=0.003)
    ax.set_title(title + f"  N={len(selected)}")
    ax.set_xlabel("x pixel")
    ax.set_ylabel("y pixel")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def true_wind_plot(frame, u_true, v_true, cfg: AMVConfig, path: Path):
    n = cfg.size
    step = cfg.step
    yy, xx = np.mgrid[step:n:step, step:n:step]
    u = u_true[yy, xx]
    v = v_true[yy, xx]
    fig, ax = plt.subplots(figsize=(8, 7))
    ax.imshow(frame, cmap="gray", origin="upper")
    ax.quiver(xx, yy, u, -v, angles="xy", scale_units="xy", scale=2200, width=0.003)
    ax.set_title("Synthetic true wind field")
    ax.set_xlabel("x pixel")
```

```
    ax.set_ylabel("y pixel")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def quality_plot(frame, rows, cfg: AMVConfig, path: Path):
    fig, ax = plt.subplots(figsize=(8, 7))
    ax.imshow(frame, cmap="gray", origin="upper")
    if rows:
        x = np.array([r["x"] for r in rows])
        y = np.array([r["y"] for r in rows])
        c = np.array([r["corr"] for r in rows])
        sc = ax.scatter(x, y, c=c, s=35, edgecolors="black", linewidths=0.25)
        fig.colorbar(sc, ax=ax, label="NCC peak correlation", shrink=0.82)
    ax.set_title("Tracking quality: peak normalised cross-correlation")
    ax.set_xlabel("x pixel")
    ax.set_ylabel("y pixel")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def height_plot(frame, rows, path: Path):
    selected = [r for r in rows if r["qc_final"]]
    fig, ax = plt.subplots(figsize=(8, 7))
    ax.imshow(frame, cmap="gray", origin="upper")
    if selected:
        x = np.array([r["x"] for r in selected])
        y = np.array([r["y"] for r in selected])
        h = np.array([r["height_km"] for r in selected])
        sc = ax.scatter(x, y, c=h, s=45, edgecolors="black", linewidths=0.25)
        fig.colorbar(sc, ax=ax, label="Assigned height km", shrink=0.82)
    ax.set_title("Height assignment for QC-passed AMVs")
    ax.set_xlabel("x pixel")
    ax.set_ylabel("y pixel")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def error_histogram(rows, path: Path):
    selected = [r for r in rows if r["qc_final"] and np.isfinite(r["vector_error_kmh"])]
    fig, ax = plt.subplots(figsize=(7, 5))
    if selected:
        e = np.array([r["vector_error_kmh"] for r in selected])
        ax.hist(e, bins=20)
        ax.axvline(float(np.median(e)), linestyle="--", label=f"median={np.median(e):.1f} km/h")
        ax.legend()
    ax.set_title("Vector error after QC")
    ax.set_xlabel("error km/h")
    ax.set_ylabel("count")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def vector_scatter(rows, path: Path):
    selected = [r for r in rows if r["qc_final"]]
    fig, ax = plt.subplots(figsize=(6, 6))
    if selected:
        u = np.array([r["u_kmh"] for r in selected])
        ut = np.array([r["u_true_kmh"] for r in selected])
        v = np.array([r["v_kmh"] for r in selected])
        vt = np.array([r["v_true_kmh"] for r in selected])
        ax.scatter(ut, u, s=25, label="u")
        ax.scatter(vt, v, s=25, label="v", marker="x")
        lo = float(min(np.min(ut), np.min(u), np.min(vt), np.min(v)))
        hi = float(max(np.max(ut), np.max(u), np.max(vt), np.max(v)))
        ax.plot([lo, hi], [lo, hi], linestyle="--")
        ax.legend()
    ax.set_title("Retrieved versus true vector components")
    ax.set_xlabel("true km/h")
    ax.set_ylabel("retrieved km/h")
    fig.tight_layout()
```

```
    fig.savefig(path, dpi=160)
    return fig


def make_animation(frame0, frame1, rows, outpath: Path):
    try:
        from PIL import Image
    except Exception:
        return False

    tmp_paths = []
    for i, frame in enumerate([frame0, frame1]):
        fig, ax = plt.subplots(figsize=(6, 6))
        ax.imshow(frame, cmap="gray", origin="upper")
        ax.set_title(f"Cloud frame {i}")
        ax.set_axis_off()
        fig.tight_layout()
        tmp = outpath.parent / f"_tmp_anim_{i}.png"
        fig.savefig(tmp, dpi=130)
        plt.close(fig)
        tmp_paths.append(tmp)

    images = [Image.open(p) for p in tmp_paths]
    images[0].save(outpath, save_all=True, append_images=images[1:], duration=650, loop=0)
    for p in tmp_paths:
        try:
            p.unlink()
        except Exception:
            pass
    return True


def write_summary(rows, cfg: AMVConfig, path: Path):
    n_raw = len(rows)
    n_basic = sum(1 for r in rows if r["qc_basic"])
    n_final = sum(1 for r in rows if r["qc_final"])
    selected = [r for r in rows if r["qc_final"] and np.isfinite(r["vector_error_kmh"])]

    lines = []
    lines.append("ATMOSPHERIC MOTION VECTOR MODEL DEMO SUMMARY")
    lines.append("=" * 55)
    lines.append(f"Image size:                {cfg.size} x {cfg.size} pixels")
    lines.append(f"Pixel scale:               {cfg.pixel_km:.2f} km/pixel")
    lines.append(f"Time step:                 {cfg.dt_minutes:.2f} minutes")
    lines.append(f"Tracking window:           {cfg.window} x {cfg.window} pixels")
    lines.append(f"Search radius:             +/- {cfg.search_radius} pixels")
    lines.append(f"Grid step:                 {cfg.step} pixels")
    lines.append("")
    lines.append(f"Raw vectors attempted:      {n_raw}")
    lines.append(f"Basic QC passed:            {n_basic}")
    lines.append(f"Final QC passed:            {n_final}")
    if selected:
        errors = np.array([r["vector_error_kmh"] for r in selected])
        corr = np.array([r["corr"] for r in selected])
        height = np.array([r["height_km"] for r in selected])
        lines.append("")
        lines.append("Final QC statistics:")
        lines.append(f"  Median vector error:      {np.median(errors):.2f} km/h")
        lines.append(f"  Mean vector error:        {np.mean(errors):.2f} km/h")
        lines.append(f"  90th percentile error:    {np.percentile(errors, 90):.2f} km/h")
        lines.append(f"  Median NCC correlation:   {np.median(corr):.3f}")
        lines.append(f"  Median assigned height:   {np.median(height):.2f} km")
    lines.append("")
    lines.append("QC tests:")
    lines.append(f"  min_corr >= {cfg.min_corr}")
    lines.append(f"  corr margin >= {cfg.min_margin}")
    lines.append(f"  speed <= {cfg.max_speed_kmh} km/h")
    lines.append(f"  neighbour consistency <= {cfg.consistency_kmh} km/h")
    lines.append("")
    lines.append("Interpretation:")
    lines.append("  This is an educational, synthetic AMV pipeline. A real operational")
    lines.append("  AMV system would need navigation calibration, channel-specific cloud")
    lines.append("  masking, radiative-transfer or NWP-assisted height assignment,")
    lines.append("  parallax correction, objective quality indicators, and validation")
    lines.append("  against radiosondes, aircraft, scatterometers, or NWP analysis winds.")
```

```
    path.write_text("\n".join(lines) + "\n")


def run(cfg: AMVConfig):
    outdir = Path(cfg.outdir)
    outdir.mkdir(parents=True, exist_ok=True)

    cloud0, bt0, height0, u_true, v_true, dx_true, dy_true = make_cloud_scene(cfg)
    cloud1, bt1 = advect_scene(cloud0, bt0, dx_true, dy_true, seed=cfg.seed + 100)

    rows = track_amvs(cloud0, cloud1, bt0, cfg)
    interpolate_truth_at_vectors(rows, u_true, v_true)

    save_csv(rows, outdir / "amv_vectors.csv")
    write_summary(rows, cfg, outdir / "run_summary.txt")

    figs = []
    figs.append(save_image(cloud0, outdir / "frame0.png", "Synthetic satellite cloud image: t0"))
    figs.append(save_image(cloud1, outdir / "frame1.png", "Synthetic satellite cloud image: t1"))
    figs.append(save_image(bt0, outdir / "brightness_temperature.png", "Synthetic brightness temperature K",
     cmap="viridis"))
    figs.append(save_image(height0, outdir / "true_height_field.png", "Synthetic true cloud-top height km",
     cmap="viridis"))
    figs.append(true_wind_plot(cloud0, u_true, v_true, cfg, outdir / "true_wind.png"))
    figs.append(quiver_plot(cloud0, rows, outdir / "retrieved_vectors_raw.png", "Retrieved AMVs before final QC
     ", final_only=False))
    figs.append(quiver_plot(cloud0, rows, outdir / "retrieved_vectors_qc.png", "Retrieved AMVs after final QC",
      final_only=True))
    figs.append(quality_plot(cloud0, rows, cfg, outdir / "tracking_quality.png"))
    figs.append(height_plot(cloud0, rows, outdir / "height_assignment.png"))
    figs.append(error_histogram(rows, outdir / "error_histogram.png"))
    figs.append(vector_scatter(rows, outdir / "vector_scatter.png"))

    if cfg.make_gif:
        make_animation(cloud0, cloud1, rows, outdir / "animation.gif")

    print("\nAMV demo complete.")
    print(f"Output directory: {outdir.resolve()}")
    print(f"CSV vectors:      {(outdir / 'amv_vectors.csv').resolve()}")
    print(f"Summary:          {(outdir / 'run_summary.txt').resolve()}")
    print("\nKey PNG files:")
    for name in [
        "frame0.png",
        "frame1.png",
        "true_wind.png",
        "retrieved_vectors_raw.png",
        "retrieved_vectors_qc.png",
        "height_assignment.png",
        "tracking_quality.png",
        "error_histogram.png",
        "vector_scatter.png",
    ]:
        print(f"  {outdir / name}")

    if cfg.show:
        for fig in figs:
            fig.show()
        plt.show()
    else:
        plt.close("all")


def parse_args():
    p = argparse.ArgumentParser(description="Synthetic Atmospheric Motion Vector model/tracking/visualisation
     demo")
    p.add_argument("--size", type=int, default=192, help="image size in pixels; try 160, 192, or 256")
    p.add_argument("--pixel-km", type=float, default=4.0, help="km per pixel")
    p.add_argument("--dt-minutes", type=float, default=10.0, help="minutes between frames")
    p.add_argument("--window", type=int, default=25, help="odd template window size")
    p.add_argument("--step", type=int, default=16, help="AMV grid spacing in pixels")
    p.add_argument("--search-radius", type=int, default=10, help="search radius in pixels")
    p.add_argument("--seed", type=int, default=42, help="random seed")
    p.add_argument("--min-corr", type=float, default=0.55, help="minimum NCC correlation")
    p.add_argument("--min-margin", type=float, default=0.015, help="minimum peak minus second peak margin")
    p.add_argument("--max-speed-kmh", type=float, default=420.0, help="reject faster winds")
```

```
    p.add_argument("--consistency-kmh", type=float, default=80.0, help="neighbour consistency threshold")
    p.add_argument("--outdir", default="output_amv", help="output directory")
    p.add_argument("--show", action="store_true", help="display figures interactively, useful in Pydroid")
    p.add_argument("--no-gif", action="store_true", help="do not create animation GIF")
    args = p.parse_args()

    size = max(80, int(args.size))
    window = ensure_odd(max(9, int(args.window)))
    if window >= size // 2:
        window = ensure_odd(size // 4)

    return AMVConfig(
        size=size,
        pixel_km=args.pixel_km,
        dt_minutes=args.dt_minutes,
        window=window,
        step=max(4, int(args.step)),
        search_radius=max(2, int(args.search_radius)),
        seed=args.seed,
        min_corr=args.min_corr,
        min_margin=args.min_margin,
        max_speed_kmh=args.max_speed_kmh,
        consistency_kmh=args.consistency_kmh,
        outdir=args.outdir,
        show=args.show,
        make_gif=not args.no_gif,
    )


if __name__ == "__main__":
    try:
        cfg = parse_args()
        run(cfg)
    except KeyboardInterrupt:
        print("\nInterrupted.", file=sys.stderr)
        raise SystemExit(130)
    except Exception as exc:
        print(f"\nERROR: {exc}", file=sys.stderr)
        raise
```

8 Source listing: one-file setup script

```
#!/usr/bin/env bash
# amv_full_onefile_setup.sh
# One-file setup, run, model, simulation and visualisation demo for
# Atmospheric Motion Vectors.
#
# Usage:
#   chmod +x amv_full_onefile_setup.sh
#   ./amv_full_onefile_setup.sh
#
# Pydroid/interactive display:
#   ./amv_full_onefile_setup.sh --show

set -eu

PROJECT_DIR="${PROJECT_DIR:-amv_demo_project}"
PYTHON_BIN="${PYTHON_BIN:-python3}"
VENV_DIR="${VENV_DIR:-.venv_amv}"

mkdir -p "$PROJECT_DIR"
cd "$PROJECT_DIR"

cat > amv_model_demo.py <<'PYTHON_AMV_DEMO'
#!/usr/bin/env python3
"""
AMV MODEL DEMO
Atmospheric Motion Vector simulation, tracking, quality control,
sub-pixel refinement, and height assignment.

This is a self-contained educational AMV demonstrator.
It does not download satellite data. It synthesizes two cloud images,
```

```
advects the cloud scene with a known wind field, then attempts to
recover motion vectors using template matching.

Outputs:
  output_amv/
    frame0.png
    frame1.png
    true_wind.png
    retrieved_vectors_raw.png
    retrieved_vectors_qc.png
    height_assignment.png
    error_histogram.png
    vector_scatter.png
    tracking_quality.png
    amv_vectors.csv
    run_summary.txt
    animation.gif      if Pillow is available

Coordinate convention:
  x increases eastward.
  y increases downward in image space.
  u is eastward wind km/h.
  v is northward wind km/h, so v = -dy * pixel_km / dt_hours.
"""

import argparse
import csv
import math
import os
import sys
from dataclasses import dataclass
from pathlib import Path

import numpy as np

import matplotlib
if "--show" not in sys.argv:
    matplotlib.use("Agg")
import matplotlib.pyplot as plt


@dataclass
class AMVConfig:
    size: int = 192
    pixel_km: float = 4.0
    dt_minutes: float = 10.0
    window: int = 25
    step: int = 16
    search_radius: int = 10
    seed: int = 42
    min_corr: float = 0.55
    min_margin: float = 0.015
    max_speed_kmh: float = 420.0
    consistency_kmh: float = 80.0
    outdir: str = "output_amv"
    show: bool = False
    make_gif: bool = True


def ensure_odd(n: int) -> int:
    return n if n % 2 == 1 else n + 1


def smooth2d(a: np.ndarray, passes: int = 5) -> np.ndarray:
    """Small dependency-free smoothing by repeated neighbour averaging."""
    out = a.astype(float).copy()
    for _ in range(passes):
        out = (
            out
            + np.roll(out, 1, axis=0)
            + np.roll(out, -1, axis=0)
            + np.roll(out, 1, axis=1)
            + np.roll(out, -1, axis=1)
            + np.roll(np.roll(out, 1, axis=0), 1, axis=1)
            + np.roll(np.roll(out, 1, axis=0), -1, axis=1)
```

```
            + np.roll(np.roll(out, -1, axis=0), 1, axis=1)
            + np.roll(np.roll(out, -1, axis=0), -1, axis=1)
        ) / 9.0
    return out


def normalize01(a: np.ndarray) -> np.ndarray:
    amin = float(np.nanmin(a))
    amax = float(np.nanmax(a))
    if amax - amin < 1e-12:
        return np.zeros_like(a, dtype=float)
    return (a - amin) / (amax - amin)


def make_cloud_scene(cfg: AMVConfig):
    rng = np.random.default_rng(cfg.seed)
    n = cfg.size
    y, x = np.mgrid[0:n, 0:n]

    # Multi-scale cloud texture.
    base = rng.normal(0.0, 1.0, (n, n))
    large = smooth2d(base, passes=22)
    medium = smooth2d(rng.normal(0.0, 1.0, (n, n)), passes=8)
    fine = smooth2d(rng.normal(0.0, 1.0, (n, n)), passes=2)
    cloud = 1.5 * large + 0.8 * medium + 0.25 * fine

    # Add Gaussian cloud shields and cellular patches.
    for _ in range(18):
        cx = rng.uniform(0, n)
        cy = rng.uniform(0, n)
        sx = rng.uniform(n * 0.035, n * 0.14)
        sy = rng.uniform(n * 0.035, n * 0.14)
        amp = rng.uniform(0.35, 1.4)
        cloud += amp * np.exp(-(((x - cx) / sx) ** 2 + ((y - cy) / sy) ** 2))

    # Add a frontal band to give elongated AMV targets.
    angle = math.radians(22.0)
    xr = (x - n * 0.5) * math.cos(angle) + (y - n * 0.5) * math.sin(angle)
    yr = -(x - n * 0.5) * math.sin(angle) + (y - n * 0.5) * math.cos(angle)
    band = np.exp(-((yr + 12 * np.sin(xr / 18.0)) / 13.0) ** 2)
    cloud += 0.8 * band

    cloud = normalize01(cloud)

    # Synthetic brightness temperature:
    # thicker/brighter cloud is colder and therefore higher.
    bt = 296.0 - 74.0 * cloud + rng.normal(0.0, 1.0, (n, n))
    height_km = np.clip((296.0 - bt) / 5.8, 0.4, 14.5)

    # True wind field in image displacement pixels per time step.
    # Higher cloud tops move faster and a little more zonally.
    hnorm = np.clip(height_km / 14.5, 0, 1)
    u_kmh = 25.0 + 115.0 * hnorm + 22.0 * np.sin(2 * np.pi * y / n)
    v_kmh = -18.0 + 55.0 * hnorm + 16.0 * np.cos(2 * np.pi * x / n)

    dt_hours = cfg.dt_minutes / 60.0
    dx_pix = u_kmh * dt_hours / cfg.pixel_km
    dy_pix = -v_kmh * dt_hours / cfg.pixel_km

    return cloud, bt, height_km, u_kmh, v_kmh, dx_pix, dy_pix


def bilinear_sample(img: np.ndarray, x: np.ndarray, y: np.ndarray, fill: float = 0.0) -> np.ndarray:
    h, w = img.shape
    x0 = np.floor(x).astype(int)
    y0 = np.floor(y).astype(int)
    x1 = x0 + 1
    y1 = y0 + 1

    wx = x - x0
    wy = y - y0

    valid = (x0 >= 0) & (x1 < w) & (y0 >= 0) & (y1 < h)
    out = np.full_like(x, fill, dtype=float)
```

```
    x0c = np.clip(x0, 0, w - 1)
    x1c = np.clip(x1, 0, w - 1)
    y0c = np.clip(y0, 0, h - 1)
    y1c = np.clip(y1, 0, h - 1)

    Ia = img[y0c, x0c]
    Ib = img[y0c, x1c]
    Ic = img[y1c, x0c]
    Id = img[y1c, x1c]

    interp = (
        Ia * (1 - wx) * (1 - wy)
        + Ib * wx * (1 - wy)
        + Ic * (1 - wx) * wy
        + Id * wx * wy
    )
    out[valid] = interp[valid]
    return out


def advect_scene(cloud0, bt0, dx_pix, dy_pix, seed=123):
    rng = np.random.default_rng(seed)
    n = cloud0.shape[0]
    yy, xx = np.mgrid[0:n, 0:n]

    # For each target pixel at time 1, sample the source pixel at time 0
    # from the approximate backward trajectory.
    src_x = xx - dx_pix
    src_y = yy - dy_pix
    fill = float(np.nanmedian(cloud0))
    cloud1 = bilinear_sample(cloud0, src_x, src_y, fill=fill)

    # Small evolution term: AMVs are harder when cloud features deform.
    evolution = 0.035 * smooth2d(rng.normal(0, 1, cloud0.shape), passes=2)
    cloud1 = normalize01(0.97 * cloud1 + evolution)

    bt1 = 296.0 - 74.0 * cloud1 + rng.normal(0.0, 1.1, cloud1.shape)
    return cloud1, bt1


def normxcorr(a: np.ndarray, b: np.ndarray) -> float:
    aa = a - float(np.mean(a))
    bb = b - float(np.mean(b))
    den = math.sqrt(float(np.sum(aa * aa)) * float(np.sum(bb * bb)))
    if den < 1e-12:
        return -1.0
    return float(np.sum(aa * bb) / den)


def parabolic_offset(c_minus: float, c0: float, c_plus: float) -> float:
    denom = c_minus - 2.0 * c0 + c_plus
    if abs(denom) < 1e-9:
        return 0.0
    delta = 0.5 * (c_minus - c_plus) / denom
    if not np.isfinite(delta):
        return 0.0
    return float(np.clip(delta, -1.0, 1.0))


def local_height_from_bt(bt: np.ndarray, x: int, y: int, half: int) -> float:
    patch = bt[y - half:y + half + 1, x - half:x + half + 1]
    # AMV height assignment often uses cold cloud information in the target.
    # Use 20th percentile brightness temperature: cold cloud top proxy.
    cold_bt = float(np.percentile(patch, 20))
    return float(np.clip((296.0 - cold_bt) / 5.8, 0.4, 14.5))


def track_amvs(frame0, frame1, bt0, cfg: AMVConfig):
    n = cfg.size
    win = ensure_odd(cfg.window)
    half = win // 2
    r = cfg.search_radius
    dt_hours = cfg.dt_minutes / 60.0

    rows = []
```

```
    centers = range(half + r, n - half - r, cfg.step)

    for cy in centers:
        for cx in centers:
            target = frame0[cy - half:cy + half + 1, cx - half:cx + half + 1]
            if float(np.std(target)) < 0.035:
                continue

            corr = np.empty((2 * r + 1, 2 * r + 1), dtype=float)
            for jj, dy in enumerate(range(-r, r + 1)):
                yy = cy + dy
                for ii, dx in enumerate(range(-r, r + 1)):
                    xx = cx + dx
                    cand = frame1[yy - half:yy + half + 1, xx - half:xx + half + 1]
                    corr[jj, ii] = normxcorr(target, cand)

            flat_index = int(np.argmax(corr))
            jmax, imax = np.unravel_index(flat_index, corr.shape)
            best = float(corr[jmax, imax])
            sorted_corr = np.sort(corr.ravel())
            second = float(sorted_corr[-2]) if sorted_corr.size >= 2 else -1.0
            margin = best - second

            # Integer displacement from frame0 centre to best frame1 centre.
            dx_int = imax - r
            dy_int = jmax - r

            # Sub-pixel correction using local parabolic peak.
            subx = 0.0
            suby = 0.0
            if 0 < imax < 2 * r:
                subx = parabolic_offset(corr[jmax, imax - 1], corr[jmax, imax], corr[jmax, imax + 1])
            if 0 < jmax < 2 * r:
                suby = parabolic_offset(corr[jmax - 1, imax], corr[jmax, imax], corr[jmax + 1, imax])

            dx = dx_int + subx
            dy = dy_int + suby

            u = dx * cfg.pixel_km / dt_hours
            v = -dy * cfg.pixel_km / dt_hours
            speed = math.sqrt(u * u + v * v)
            height = local_height_from_bt(bt0, cx, cy, half)

            qc_basic = (
                best >= cfg.min_corr
                and margin >= cfg.min_margin
                and speed <= cfg.max_speed_kmh
            )

            rows.append({
                "x": float(cx),
                "y": float(cy),
                "dx_pix": float(dx),
                "dy_pix": float(dy),
                "u_kmh": float(u),
                "v_kmh": float(v),
                "speed_kmh": float(speed),
                "corr": float(best),
                "corr_margin": float(margin),
                "height_km": float(height),
                "qc_basic": bool(qc_basic),
                "qc_consistency": False,
                "qc_final": False,
                "du_error_kmh": np.nan,
                "dv_error_kmh": np.nan,
                "vector_error_kmh": np.nan,
            })

    apply_consistency_qc(rows, cfg)
    return rows


def apply_consistency_qc(rows, cfg: AMVConfig):
    if not rows:
        return
```

```
    coords = np.array([[r["x"], r["y"]] for r in rows], dtype=float)
    u = np.array([r["u_kmh"] for r in rows], dtype=float)
    v = np.array([r["v_kmh"] for r in rows], dtype=float)
    basic = np.array([r["qc_basic"] for r in rows], dtype=bool)

    for i, row in enumerate(rows):
        if not row["qc_basic"]:
            row["qc_consistency"] = False
            row["qc_final"] = False
            continue

        dist = np.sqrt(np.sum((coords - coords[i]) ** 2, axis=1))
        nbr = (dist > 0) & (dist <= cfg.step * 1.6) & basic
        if np.count_nonzero(nbr) < 3:
            row["qc_consistency"] = True
            row["qc_final"] = True
            continue

        med_u = float(np.median(u[nbr]))
        med_v = float(np.median(v[nbr]))
        diff = math.sqrt((row["u_kmh"] - med_u) ** 2 + (row["v_kmh"] - med_v) ** 2)

        row["qc_consistency"] = bool(diff <= cfg.consistency_kmh)
        row["qc_final"] = bool(row["qc_basic"] and row["qc_consistency"])


def interpolate_truth_at_vectors(rows, u_true, v_true):
    n = u_true.shape[0]
    for row in rows:
        x = int(np.clip(round(row["x"]), 0, n - 1))
        y = int(np.clip(round(row["y"]), 0, n - 1))
        du = row["u_kmh"] - float(u_true[y, x])
        dv = row["v_kmh"] - float(v_true[y, x])
        row["u_true_kmh"] = float(u_true[y, x])
        row["v_true_kmh"] = float(v_true[y, x])
        row["du_error_kmh"] = float(du)
        row["dv_error_kmh"] = float(dv)
        row["vector_error_kmh"] = float(math.sqrt(du * du + dv * dv))


def rows_to_arrays(rows, final_only=False):
    selected = [r for r in rows if (r["qc_final"] or not final_only)]
    if not selected:
        return {}
    return {k: np.array([r[k] for r in selected]) for k in selected[0].keys()}


def save_csv(rows, path: Path):
    if not rows:
        return
    fieldnames = list(rows[0].keys())
    with path.open("w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        for r in rows:
            writer.writerow(r)


def save_image(img, path: Path, title: str, cmap="gray", colorbar=True):
    fig, ax = plt.subplots(figsize=(7, 6))
    im = ax.imshow(img, cmap=cmap, origin="upper")
    ax.set_title(title)
    ax.set_xlabel("x pixel")
    ax.set_ylabel("y pixel")
    if colorbar:
        fig.colorbar(im, ax=ax, shrink=0.82)
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def quiver_plot(frame, rows, path: Path, title: str, final_only=False, scale=2200):
    selected = [r for r in rows if (r["qc_final"] or not final_only)]
    fig, ax = plt.subplots(figsize=(8, 7))
```

```
    ax.imshow(frame, cmap="gray", origin="upper")
    if selected:
        x = np.array([r["x"] for r in selected])
        y = np.array([r["y"] for r in selected])
        u = np.array([r["u_kmh"] for r in selected])
        v = np.array([r["v_kmh"] for r in selected])
        # Convert north-positive v back to image-axis vector component for display.
        ax.quiver(x, y, u, -v, angles="xy", scale_units="xy", scale=scale, width=0.003)
    ax.set_title(title + f"  N={len(selected)}")
    ax.set_xlabel("x pixel")
    ax.set_ylabel("y pixel")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def true_wind_plot(frame, u_true, v_true, cfg: AMVConfig, path: Path):
    n = cfg.size
    step = cfg.step
    yy, xx = np.mgrid[step:n:step, step:n:step]
    u = u_true[yy, xx]
    v = v_true[yy, xx]
    fig, ax = plt.subplots(figsize=(8, 7))
    ax.imshow(frame, cmap="gray", origin="upper")
    ax.quiver(xx, yy, u, -v, angles="xy", scale_units="xy", scale=2200, width=0.003)
    ax.set_title("Synthetic true wind field")
    ax.set_xlabel("x pixel")
    ax.set_ylabel("y pixel")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def quality_plot(frame, rows, cfg: AMVConfig, path: Path):
    fig, ax = plt.subplots(figsize=(8, 7))
    ax.imshow(frame, cmap="gray", origin="upper")
    if rows:
        x = np.array([r["x"] for r in rows])
        y = np.array([r["y"] for r in rows])
        c = np.array([r["corr"] for r in rows])
        sc = ax.scatter(x, y, c=c, s=35, edgecolors="black", linewidths=0.25)
        fig.colorbar(sc, ax=ax, label="NCC peak correlation", shrink=0.82)
    ax.set_title("Tracking quality: peak normalised cross-correlation")
    ax.set_xlabel("x pixel")
    ax.set_ylabel("y pixel")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def height_plot(frame, rows, path: Path):
    selected = [r for r in rows if r["qc_final"]]
    fig, ax = plt.subplots(figsize=(8, 7))
    ax.imshow(frame, cmap="gray", origin="upper")
    if selected:
        x = np.array([r["x"] for r in selected])
        y = np.array([r["y"] for r in selected])
        h = np.array([r["height_km"] for r in selected])
        sc = ax.scatter(x, y, c=h, s=45, edgecolors="black", linewidths=0.25)
        fig.colorbar(sc, ax=ax, label="Assigned height km", shrink=0.82)
    ax.set_title("Height assignment for QC-passed AMVs")
    ax.set_xlabel("x pixel")
    ax.set_ylabel("y pixel")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def error_histogram(rows, path: Path):
    selected = [r for r in rows if r["qc_final"] and np.isfinite(r["vector_error_kmh"])]
    fig, ax = plt.subplots(figsize=(7, 5))
    if selected:
        e = np.array([r["vector_error_kmh"] for r in selected])
        ax.hist(e, bins=20)
        ax.axvline(float(np.median(e)), linestyle="--", label=f"median={np.median(e):.1f} km/h")
```

```
        ax.legend()
    ax.set_title("Vector error after QC")
    ax.set_xlabel("error km/h")
    ax.set_ylabel("count")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def vector_scatter(rows, path: Path):
    selected = [r for r in rows if r["qc_final"]]
    fig, ax = plt.subplots(figsize=(6, 6))
    if selected:
        u = np.array([r["u_kmh"] for r in selected])
        ut = np.array([r["u_true_kmh"] for r in selected])
        v = np.array([r["v_kmh"] for r in selected])
        vt = np.array([r["v_true_kmh"] for r in selected])
        ax.scatter(ut, u, s=25, label="u")
        ax.scatter(vt, v, s=25, label="v", marker="x")
        lo = float(min(np.min(ut), np.min(u), np.min(vt), np.min(v)))
        hi = float(max(np.max(ut), np.max(u), np.max(vt), np.max(v)))
        ax.plot([lo, hi], [lo, hi], linestyle="--")
        ax.legend()
    ax.set_title("Retrieved versus true vector components")
    ax.set_xlabel("true km/h")
    ax.set_ylabel("retrieved km/h")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    return fig


def make_animation(frame0, frame1, rows, outpath: Path):
    try:
        from PIL import Image
    except Exception:
        return False

    tmp_paths = []
    for i, frame in enumerate([frame0, frame1]):
        fig, ax = plt.subplots(figsize=(6, 6))
        ax.imshow(frame, cmap="gray", origin="upper")
        ax.set_title(f"Cloud frame {i}")
        ax.set_axis_off()
        fig.tight_layout()
        tmp = outpath.parent / f"_tmp_anim_{i}.png"
        fig.savefig(tmp, dpi=130)
        plt.close(fig)
        tmp_paths.append(tmp)

    images = [Image.open(p) for p in tmp_paths]
    images[0].save(outpath, save_all=True, append_images=images[1:], duration=650, loop=0)
    for p in tmp_paths:
        try:
            p.unlink()
        except Exception:
            pass
    return True


def write_summary(rows, cfg: AMVConfig, path: Path):
    n_raw = len(rows)
    n_basic = sum(1 for r in rows if r["qc_basic"])
    n_final = sum(1 for r in rows if r["qc_final"])
    selected = [r for r in rows if r["qc_final"] and np.isfinite(r["vector_error_kmh"])]

    lines = []
    lines.append("ATMOSPHERIC MOTION VECTOR MODEL DEMO SUMMARY")
    lines.append("=" * 55)
    lines.append(f"Image size:                {cfg.size} x {cfg.size} pixels")
    lines.append(f"Pixel scale:               {cfg.pixel_km:.2f} km/pixel")
    lines.append(f"Time step:                 {cfg.dt_minutes:.2f} minutes")
    lines.append(f"Tracking window:           {cfg.window} x {cfg.window} pixels")
    lines.append(f"Search radius:             +/- {cfg.search_radius} pixels")
    lines.append(f"Grid step:                 {cfg.step} pixels")
    lines.append("")
```

```
    lines.append(f"Raw vectors attempted:       {n_raw}")
    lines.append(f"Basic QC passed:             {n_basic}")
    lines.append(f"Final QC passed:             {n_final}")
    if selected:
        errors = np.array([r["vector_error_kmh"] for r in selected])
        corr = np.array([r["corr"] for r in selected])
        height = np.array([r["height_km"] for r in selected])
        lines.append("")
        lines.append("Final QC statistics:")
        lines.append(f"  Median vector error:       {np.median(errors):.2f} km/h")
        lines.append(f"  Mean vector error:         {np.mean(errors):.2f} km/h")
        lines.append(f"  90th percentile error:     {np.percentile(errors, 90):.2f} km/h")
        lines.append(f"  Median NCC correlation:    {np.median(corr):.3f}")
        lines.append(f"  Median assigned height:    {np.median(height):.2f} km")
    lines.append("")
    lines.append("QC tests:")
    lines.append(f"  min_corr >= {cfg.min_corr}")
    lines.append(f"  corr margin >= {cfg.min_margin}")
    lines.append(f"  speed <= {cfg.max_speed_kmh} km/h")
    lines.append(f"  neighbour consistency <= {cfg.consistency_kmh} km/h")
    lines.append("")
    lines.append("Interpretation:")
    lines.append("  This is an educational, synthetic AMV pipeline. A real operational")
    lines.append("  AMV system would need navigation calibration, channel-specific cloud")
    lines.append("  masking, radiative-transfer or NWP-assisted height assignment,")
    lines.append("  parallax correction, objective quality indicators, and validation")
    lines.append("  against radiosondes, aircraft, scatterometers, or NWP analysis winds.")
    path.write_text("\n".join(lines) + "\n")


def run(cfg: AMVConfig):
    outdir = Path(cfg.outdir)
    outdir.mkdir(parents=True, exist_ok=True)

    cloud0, bt0, height0, u_true, v_true, dx_true, dy_true = make_cloud_scene(cfg)
    cloud1, bt1 = advect_scene(cloud0, bt0, dx_true, dy_true, seed=cfg.seed + 100)

    rows = track_amvs(cloud0, cloud1, bt0, cfg)
    interpolate_truth_at_vectors(rows, u_true, v_true)

    save_csv(rows, outdir / "amv_vectors.csv")
    write_summary(rows, cfg, outdir / "run_summary.txt")

    figs = []
    figs.append(save_image(cloud0, outdir / "frame0.png", "Synthetic satellite cloud image: t0"))
    figs.append(save_image(cloud1, outdir / "frame1.png", "Synthetic satellite cloud image: t1"))
    figs.append(save_image(bt0, outdir / "brightness_temperature.png", "Synthetic brightness temperature K",
     cmap="viridis"))
    figs.append(save_image(height0, outdir / "true_height_field.png", "Synthetic true cloud-top height km",
     cmap="viridis"))
    figs.append(true_wind_plot(cloud0, u_true, v_true, cfg, outdir / "true_wind.png"))
    figs.append(quiver_plot(cloud0, rows, outdir / "retrieved_vectors_raw.png", "Retrieved AMVs before final QC
     ", final_only=False))
    figs.append(quiver_plot(cloud0, rows, outdir / "retrieved_vectors_qc.png", "Retrieved AMVs after final QC",
      final_only=True))
    figs.append(quality_plot(cloud0, rows, cfg, outdir / "tracking_quality.png"))
    figs.append(height_plot(cloud0, rows, outdir / "height_assignment.png"))
    figs.append(error_histogram(rows, outdir / "error_histogram.png"))
    figs.append(vector_scatter(rows, outdir / "vector_scatter.png"))

    if cfg.make_gif:
        make_animation(cloud0, cloud1, rows, outdir / "animation.gif")

    print("\nAMV demo complete.")
    print(f"Output directory: {outdir.resolve()}")
    print(f"CSV vectors:      {(outdir / 'amv_vectors.csv').resolve()}")
    print(f"Summary:          {(outdir / 'run_summary.txt').resolve()}")
    print("\nKey PNG files:")
    for name in [
        "frame0.png",
        "frame1.png",
        "true_wind.png",
        "retrieved_vectors_raw.png",
        "retrieved_vectors_qc.png",
        "height_assignment.png",
```

```
        "tracking_quality.png",
        "error_histogram.png",
        "vector_scatter.png",
    ]:
        print(f"  {outdir / name}")

    if cfg.show:
        for fig in figs:
            fig.show()
        plt.show()
    else:
        plt.close("all")


def parse_args():
    p = argparse.ArgumentParser(description="Synthetic Atmospheric Motion Vector model/tracking/visualisation
     demo")
    p.add_argument("--size", type=int, default=192, help="image size in pixels; try 160, 192, or 256")
    p.add_argument("--pixel-km", type=float, default=4.0, help="km per pixel")
    p.add_argument("--dt-minutes", type=float, default=10.0, help="minutes between frames")
    p.add_argument("--window", type=int, default=25, help="odd template window size")
    p.add_argument("--step", type=int, default=16, help="AMV grid spacing in pixels")
    p.add_argument("--search-radius", type=int, default=10, help="search radius in pixels")
    p.add_argument("--seed", type=int, default=42, help="random seed")
    p.add_argument("--min-corr", type=float, default=0.55, help="minimum NCC correlation")
    p.add_argument("--min-margin", type=float, default=0.015, help="minimum peak minus second peak margin")
    p.add_argument("--max-speed-kmh", type=float, default=420.0, help="reject faster winds")
    p.add_argument("--consistency-kmh", type=float, default=80.0, help="neighbour consistency threshold")
    p.add_argument("--outdir", default="output_amv", help="output directory")
    p.add_argument("--show", action="store_true", help="display figures interactively, useful in Pydroid")
    p.add_argument("--no-gif", action="store_true", help="do not create animation GIF")
    args = p.parse_args()

    size = max(80, int(args.size))
    window = ensure_odd(max(9, int(args.window)))
    if window >= size // 2:
        window = ensure_odd(size // 4)

    return AMVConfig(
        size=size,
        pixel_km=args.pixel_km,
        dt_minutes=args.dt_minutes,
        window=window,
        step=max(4, int(args.step)),
        search_radius=max(2, int(args.search_radius)),
        seed=args.seed,
        min_corr=args.min_corr,
        min_margin=args.min_margin,
        max_speed_kmh=args.max_speed_kmh,
        consistency_kmh=args.consistency_kmh,
        outdir=args.outdir,
        show=args.show,
        make_gif=not args.no_gif,
    )


if __name__ == "__main__":
    try:
        cfg = parse_args()
        run(cfg)
    except KeyboardInterrupt:
        print("\nInterrupted.", file=sys.stderr)
        raise SystemExit(130)
    except Exception as exc:
        print(f"\nERROR: {exc}", file=sys.stderr)
        raise

PYTHON_AMV_DEMO

echo "Atmospheric Motion Vector demo setup"
echo "Project directory: $(pwd)"

if ! command -v "$PYTHON_BIN" >/dev/null 2>&1; then
    echo "ERROR: $PYTHON_BIN not found. Install Python 3 first." >&2
    exit 1
```

```
fi

if "$PYTHON_BIN" -m venv "$VENV_DIR" >/dev/null 2>&1; then
    # shellcheck disable=SC1091
    . "$VENV_DIR/bin/activate"
    PY="$VENV_DIR/bin/python"
else
    echo "WARNING: Could not create a virtual environment."
    echo "Falling back to --user package installation."
    PY="$PYTHON_BIN"
fi

"$PY" -m pip install --upgrade pip
"$PY" -m pip install numpy matplotlib pandas pillow

echo
echo "Running AMV model demo..."
"$PY" amv_model_demo.py "$@"

echo
echo "Done. Inspect:"
echo "  cd $PROJECT_DIR"
echo "  ls -lh output_amv"
echo "  cat output_amv/run_summary.txt"
```

9 Source listing: setup script

```
#!/usr/bin/env bash
# setup_run_amv_demo.sh
# End-to-end setup and execution script for the Atmospheric Motion Vector demo.
#
# Usage:
#   chmod +x setup_run_amv_demo.sh
#   ./setup_run_amv_demo.sh
#
# Pydroid/interactive display:
#   ./setup_run_amv_demo.sh --show
#
# Smaller/faster run:
#   ./setup_run_amv_demo.sh --size 128 --step 16 --window 21 --search-radius 8
#
# The Python model is in amv_model_demo.py.

set -eu

PYTHON_BIN="${PYTHON_BIN:-python3}"
VENV_DIR="${VENV_DIR:-.venv_amv}"
SCRIPT_DIR="$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)"
cd "$SCRIPT_DIR"

echo "Atmospheric Motion Vector demo setup"
echo "Working directory: $SCRIPT_DIR"

if ! command -v "$PYTHON_BIN" >/dev/null 2>&1; then
    echo "ERROR: $PYTHON_BIN not found. Install Python 3 first." >&2
    exit 1
fi

if [ ! -f "amv_model_demo.py" ]; then
    echo "ERROR: amv_model_demo.py not found in $SCRIPT_DIR" >&2
    exit 1
fi

# Try a virtual environment first. If venv is unavailable, fall back to user install.
USE_VENV=0
if "$PYTHON_BIN" -m venv "$VENV_DIR" >/dev/null 2>&1; then
    USE_VENV=1
    # shellcheck disable=SC1091
    . "$VENV_DIR/bin/activate"
    PY="$VENV_DIR/bin/python"
else
    echo "WARNING: Could not create a virtual environment."
```

```
    echo "Falling back to --user package installation."
    PY="$PYTHON_BIN"
fi

echo "Python executable: $PY"
"$PY" -m pip install --upgrade pip
"$PY" -m pip install numpy matplotlib pandas pillow

echo
echo "Running AMV model demo..."
"$PY" amv_model_demo.py "$@"

echo
echo "Done. Inspect the output_amv directory:"
echo "  ls -lh output_amv"
echo "  cat output_amv/run_summary.txt"
```

10 README

```
# Atmospheric Motion Vector Synthetic Model Demo

Files:
- `setup_run_amv_demo.sh`: installs dependencies and runs the demo.
- `amv_model_demo.py`: complete Python AMV simulator/tracker/visualiser.

Run:

```bash
chmod +x setup_run_amv_demo.sh
./setup_run_amv_demo.sh
```

Interactive display, useful in Pydroid:

```bash
./setup_run_amv_demo.sh --show
```

Outputs go to `output_amv/`, including PNG visualisations, CSV vectors, and a run summary.

Main features:
- synthetic cloud image simulation
- known true wind field
- frame-to-frame advection
- normalised cross-correlation template tracking
- sub-pixel peak refinement
- quality control
- brightness-temperature cloud-top height assignment
- CSV and PNG outputs
```

11 Reproducibility

To rerun the project from this report package, use:

```
cd amv_latex_report
bash src/amv_full_onefile_setup.sh --size 128 --step 16 --window 21 --search-radius 8
```

For an interactive Pydroid-style display mode, append `--show`. On Android environments, installing `numpy`, `matplotlib`, `pandas`, and `pillow` through the local Python package manager may be required.

12 Limitations and scientific critique

This is a controlled demonstration, not an operational AMV production system. Operational AMV retrieval normally requires calibrated satellite radiances, navigation/geolocation metadata, paral-

lax handling, cloud-mask classification, multi-channel height assignment, pressure-level assignment, background-model checks, and far more stringent quality indicators. The present model is nevertheless useful as a transparent teaching and prototyping tool because the true wind field is known and the full error structure can be visualised.